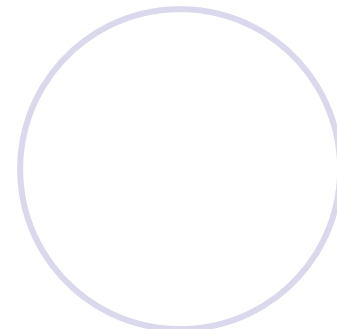
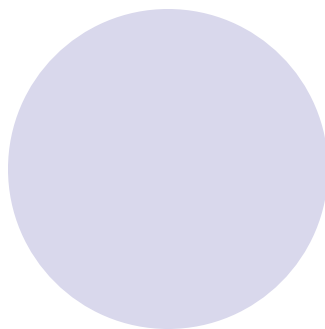
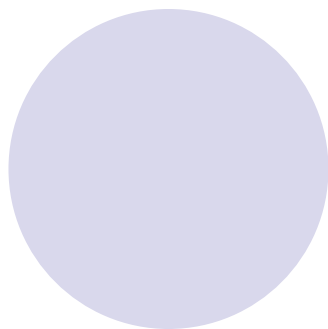
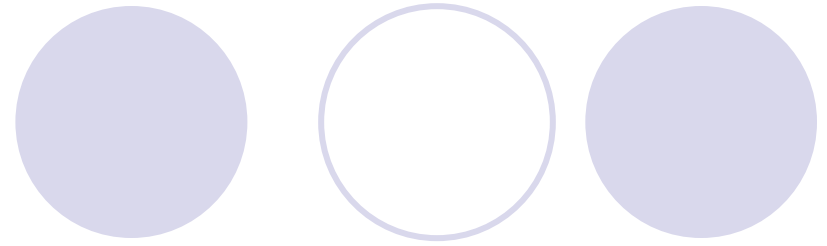


区间中的动态规划



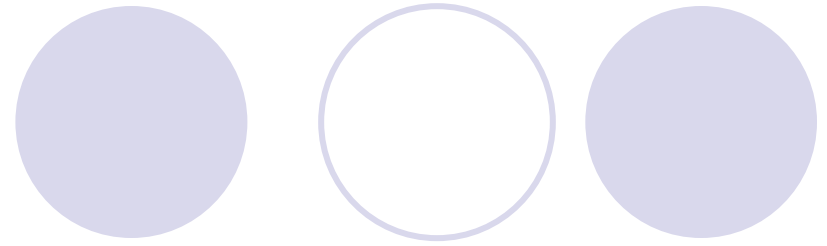
- 
- 区间**DP**是一类在区间上进行动态规划的最优问题，一般是根据问题设出一个表示状态的**dp**，一般情况下为二维，当然情况也可以是三维。
 - 一般策略：将问题划分成两个子问题，也就是一段区间分成左右两个区间，然后将左右两个区间合并到整个区间，或者说局部最优解合并为全局最优解，然后得解。

最大子段和



- 给你一个长度为N的数组，找出一对 $l \leq r$ 使得 $\text{sum}(a[l], a[l+1], \dots, a[r])$ 最大，求这个最值。
- 1 4 -2 3 -6 5
- $l=0$ $r=3$ 时， $\text{sum}=6$ ， 最大

算法一： $O(n^2)$



- 枚举区间起点 l ，终点 r ，暴力求解得到所有的 $[l, r]$ 的和，然后求解其中的最大值；
- 正确性肯定，问题是如果不超时的话，那么 n 最大是1000~10000；

算法二： $O(n \log n)$

把区间二分，那么最大子段，可能的情况：

1. 完全在左区间
2. 完全在右区间
3. 子段跨越左右区间

该如何实现了？

分治

更优的解法: $O(n)$

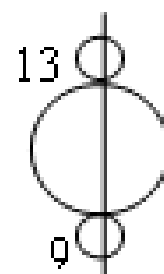
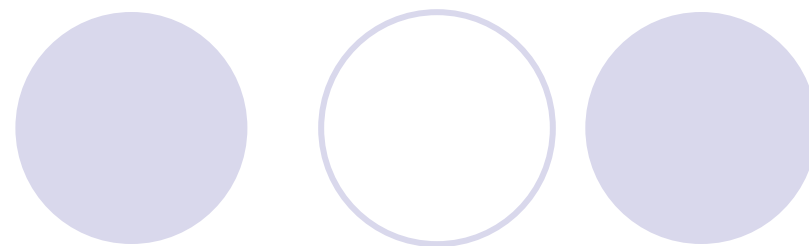
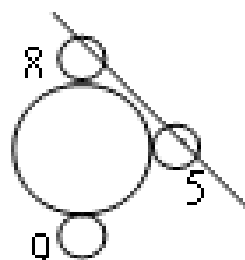
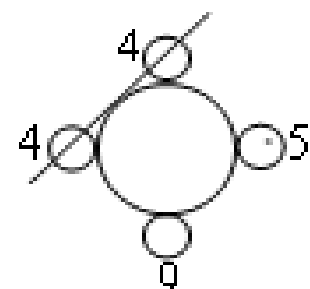
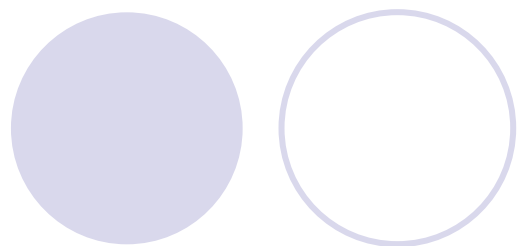
- 定义数组 $dp[i]$ 表示以 i 为最右边 r 时最大的 sum 值

$A[i]: 1 \ 5 \ -8 \ 3 \ -5 \ 5$

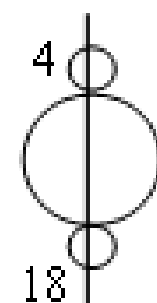
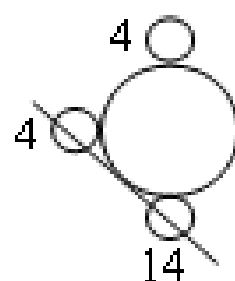
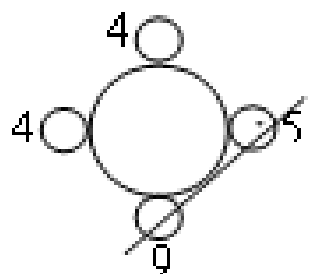
- 如何计算?
- $dp[i] = \max(a[i], dp[i-1] + a[i]);$

石子合并

- 在一个圆形操场的四周摆放着 n 堆石子。现要将石子有次序地合并成一堆。规定每次只能选相邻的**2**堆石子合并成新的一堆，并将新的一堆石子数记为该次合并的得分。试设计一个算法，计算出将 n 堆石子合并成一堆的最小得分。



总得分=8+13+22=43



总得分=14+18+22=54

贪心法

N=5 石子数分别为3 4 6 5 4 2。

用贪心法的合并过程如下：

第一次 3 4 6 5 4 2得分 5

第二次 5 4 6 5 4得分9

第三次 9 6 5 4得分9

第四次 9 6 9得分15

第五次 15 9得分24

第六次24

总分： 62

然而仔细琢磨后，发现更好的方案：

第一次3 4 6 5 4 2得分 7

第二次7 6 5 4 2得分13

第三次13 5 4 2得分6

第四次13 5 6得分11

第五次 13 11得分24

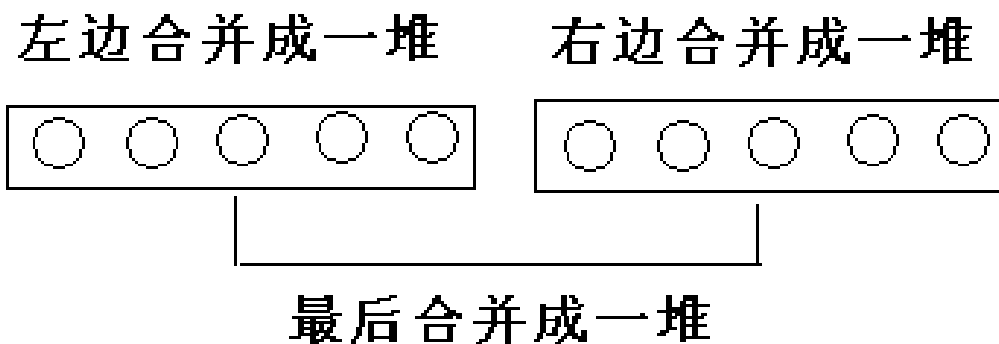
第六次24

总分： 61

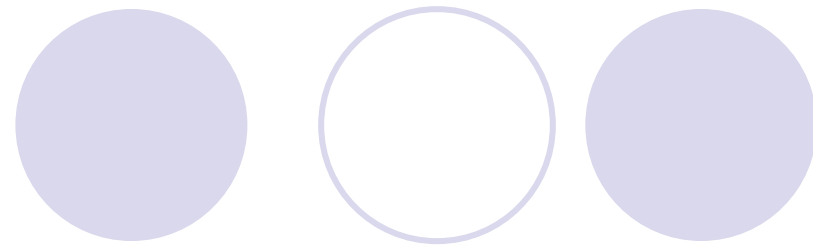
显然，贪心法是错误的。

分析

- 先只考虑直线的情况
- 假设只有**2**堆石子，显然只有**1**种合并方案
- 如果有**3**堆石子，则有**2**种合并方案， $((1,2),3)$ 和 $(1,(2,3))$
- 如果有**4**堆石子，则有**5**种合并方案
- $((1,2),3),4$ 、 $((1,(2,3)),4)$ 、 $(1,((2,3),4))$ 、 $(1,(2,(3,4)))$ 、 $((1,2),(3,4))$
- 如果有**k**堆石子呢？
- 不管怎么合并，总之最后总会归结为**2**堆，如果我们把最后两堆分开，左边和右边无论怎么合并，都必须满足最优合并方案，整个问题才能得到最优解。如下图：

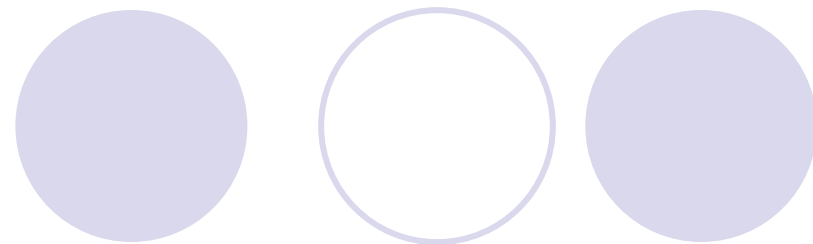


本题属于决策类dp

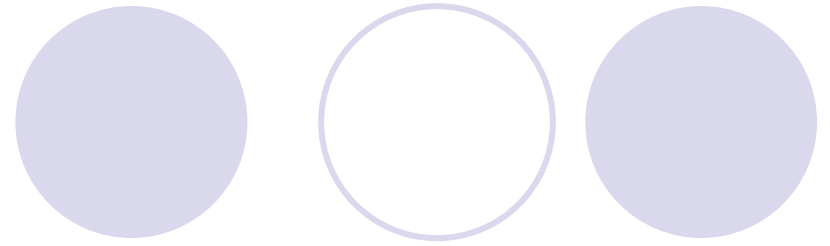
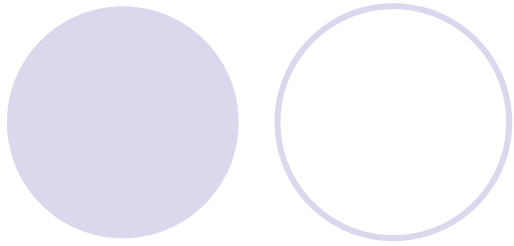


- 递推
- 程序框架（从边界开始，分阶段确定当前状态）
 - 边界初始化
 - 枚举所有阶段
 - 枚举当前阶段所有状态
 - 枚举到达当前状态的所有最后一次决策
 - 找最优决策，确定当前状态

1048石子归并



- 设 $f[i][j]$: 区间 $[i,j]$ 合并成一堆的最优值
- $f[i][j]=f[i][k]+f[k+1][j]+cost[i,j]$
- 阶段: $len: 2 \rightarrow n$
- 状态: $i: 1 \rightarrow n - len + 1, j: i + len - 1$
- 策略: $k: i \rightarrow j$
- 所以时间复杂度为 n^4
- 优化: 预处理 $sum[i]$ 表示前 i 堆石子总数量, 则 $cost[i,j] = sum[j] - sum[i - 1]$, 时间优化为 n^3
- 代码演示:



- 加上环怎么办？

- 断环为链：将长度为 n 的链复制一份接在后面，环的情况就是长度为 $2n$ 的链中任意连续的长度为 n 的链。

例三：codevs 1154 能量项链

- 分析：首先项链是环状的，我们肯定要把它从某处断开，从哪断开呢？枚举咯。
- 当我们已经把它从某处断开以后，将断开后的珠子按顺序依次标号为1~n，这时如果我们用 $dp[i][j]$ 表示从第i颗珠子到第j颗珠子合并所能得到的最大能量，则所求即为 $dp[1][n]$ 。
- 如果再用 $e[i]$ 表示第i颗珠子的头标记，那么容易发现状态转移方程：
$$dp[i][j] = \max\{dp[i][k] + dp[k+1][j] + e[i] * e[k+1] * e[j+1]\} \mid (i \leq k < j)。$$

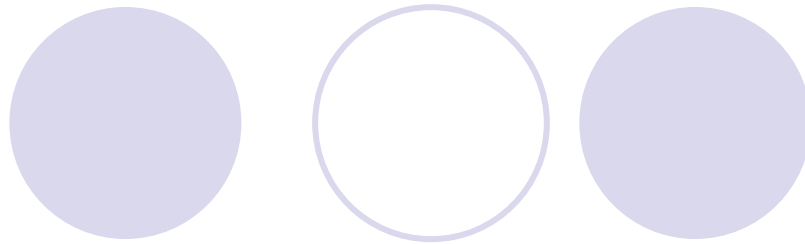
- 这样，枚举断开点、 i 、 j 、 k ，复杂度为 $O(n^4)$ 。这对于 $n \leq 100$ 的数据是不能承受的。那么复杂度再能不能降了呢？
- 我们可以在读入数据时把 $e[]$ 复制一份，像这样：
`cin>>e[i];e[i+n]=e[i]`。这样断环为链后就不需要枚举断开点了，因为当断开点为 n 或 1 时，其对应 $dp[1][n]$ ；当断开点为 1 或 2 时，其对应 $dp[2][n+1]$ ；以此类推。复杂度便降为 $O(n^3)$

```
for(int j=2;j<n+n;j++)
    for(int i=j-1;i>=1;i--)
    {
        for(int k=i;k<j && j-i<n;k++)//保证求得的始终是原项链
        {
            dp[i][j]=max(dp[i][j],dp[i][k]+dp[k+1][j]+a[i]*a[k+1]*a[j+1]);
        }
        if(ans<dp[i][j])ans=dp[i][j];
    }
```

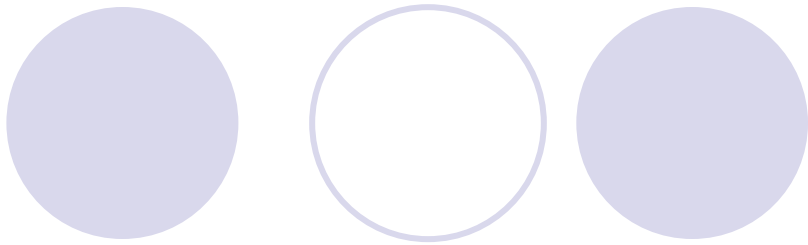
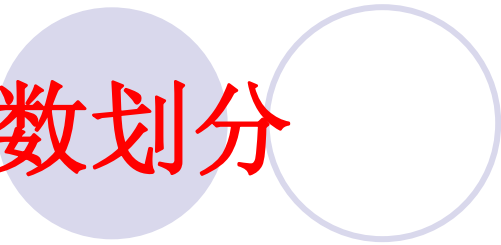
作业

- Codevs 1048
- Codevs 2102
- Codevs 1154

区间类划分



整数划分



- 给出一个长度为 n 的数
- 要在其中加 $m-1$ 个乘号，分成 m 段
- 这 m 段的乘积之和最大
- $m < n \leq 20$
- 有 T 组数据， $T \leq 10000$

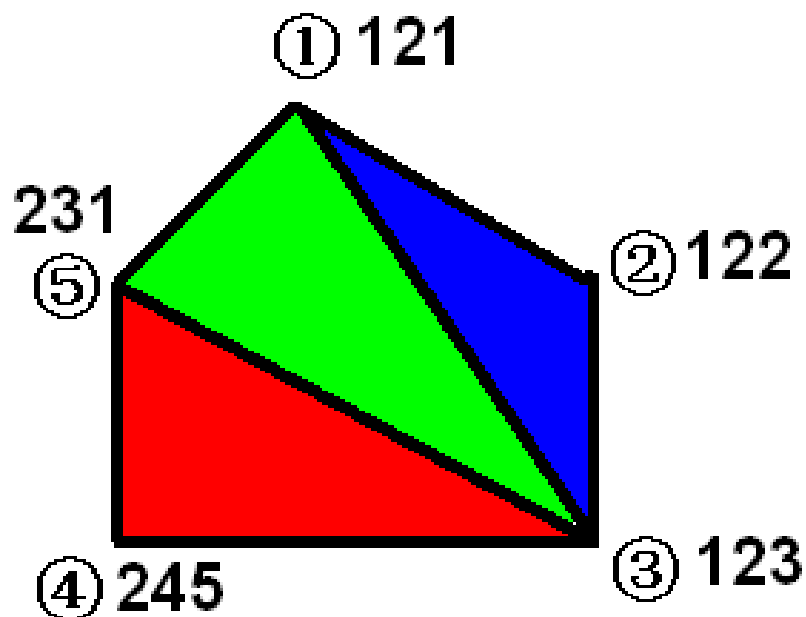
动态规划

- 可以先预处理出原数第*i*到*j*段的数值 **$A[i][j]$** 是多少，这样转移就方便了，预处理也要尽量降低复杂度。
- **$F[i][j]$** 表示把这个数前*i*位分成*j*段得到的最大乘积。
- **$F[i][j]=F[k][j-1]*A[k+1][i]$** ,
- **$1 < k < i \leq n, j \leq m$**
- 时间复杂度为 **$O[m^2n]$**
- 由于有**10000**组数据，因此估计时间复杂度为 **$10000*20^3=8*10^7$**
- 至于说输出，记录转移的父亲就可以了。

凸多边形的三角剖分

- 给定由 N 顶点组成的凸多边形
- 每个顶点具有权值
- 将凸 N 边形剖分成 $N-2$ 个三角形
- 求 $N-2$ 个三角形顶点权值乘积之和最小？

● 样例



上述凸五边形分成 $\triangle 123$ ， $\triangle 135$ ， $\triangle 345$
三角形顶点权值乘积之和为：

$$121*122*123+121*123*231+123*245*231=12214884$$

分析

- 性质：一个凸多边形剖分一个三角形后，可以将凸多边形剖分成三个部分：
 - ◆ 一个三角形
 - ◆ 二个凸多边形（图2可以看成另一个凸多边形为0）

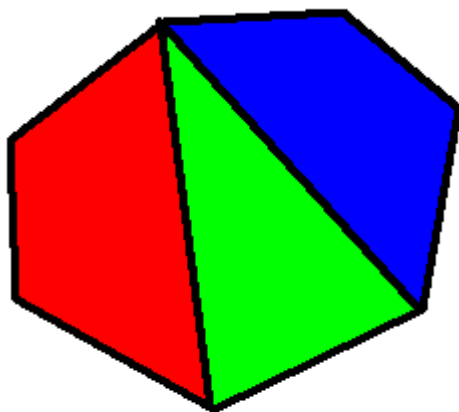


图1

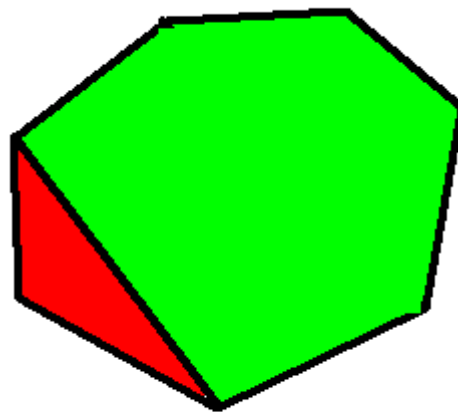
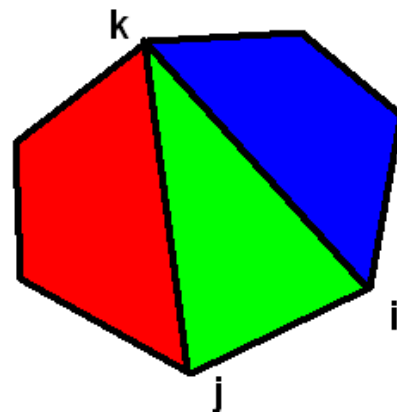


图2

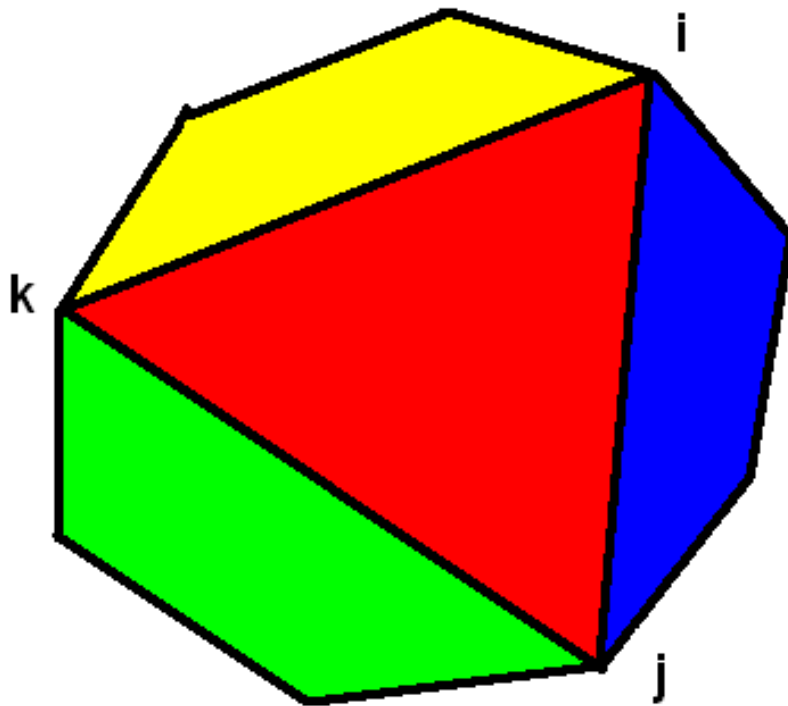
动态规划

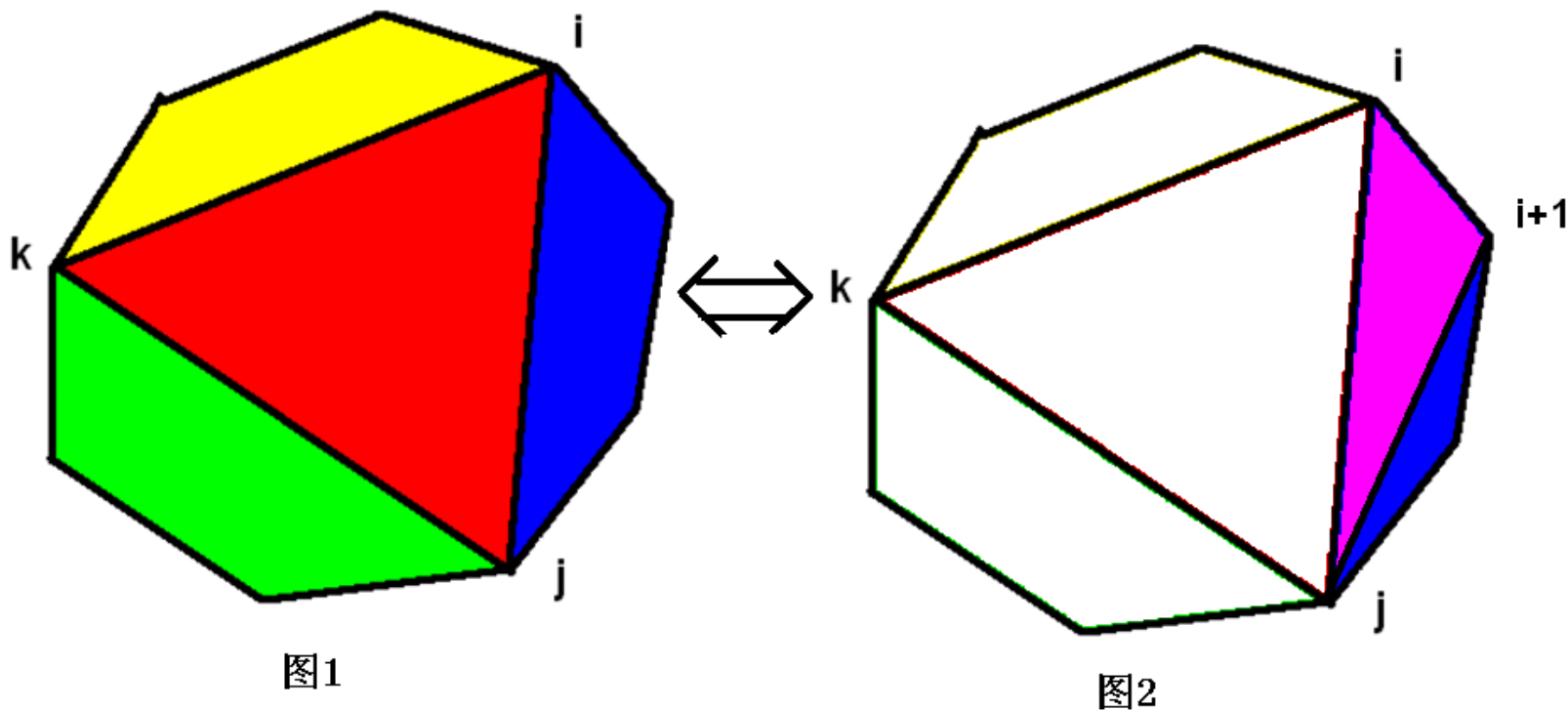
- 如果我们按顺时针将顶点编号，则可以相邻两个顶点描述一个凸多边形。
- 设 $f(i,j)$ 表示 $i \sim j$ 这一段连续顶点的多边形划分后最小乘积
- 枚举点 k ， i 、 j 和 k 相连成基本三角形，并把原多边形划分成两个子多边形，则有
- $f(i,j) = \min\{f(i,k) + f(k,j) + a[i] * a[j] * a[k]\} \quad (j-i \geq 2, f(i,i+1) = 0)$
- $1 \leq i < k < j \leq n$
- 答案为 $f(1,n)$
- 时间复杂度 $O(n^3)$



讨论

➤ 为什么可以不考虑这种情况？





- 可以看出图1和图2是等价的，也就是说如果存在图1的剖分方案，则可以转化成图2的剖分方案，因此可以不考虑图1的这种情形。